CHAPTER 0

Getting Started: Software, Information Systems & Python

Your first step into the world of code

0

3-hour session · Theory + Live Practice + Lab

What we'll cover today

1. What software and software engineering are
2. What an information system is, and where data fits in
3. Why Python is a great first language
4. Variables, print(), input(), and basic data types
5. Making decisions with if / else
6. Repeating actions with for loops
7. Storing data with lists, tuples, and dictionaries
8. Using libraries with import — and building a real game!



LET'S DISCUSS

What comes to mind when you hear the word “software”?

Think about your phone, your laptop, even this screen.

What is Software?



Application software

Programs you use directly: Instagram, Word, a calculator app, a game.



System software

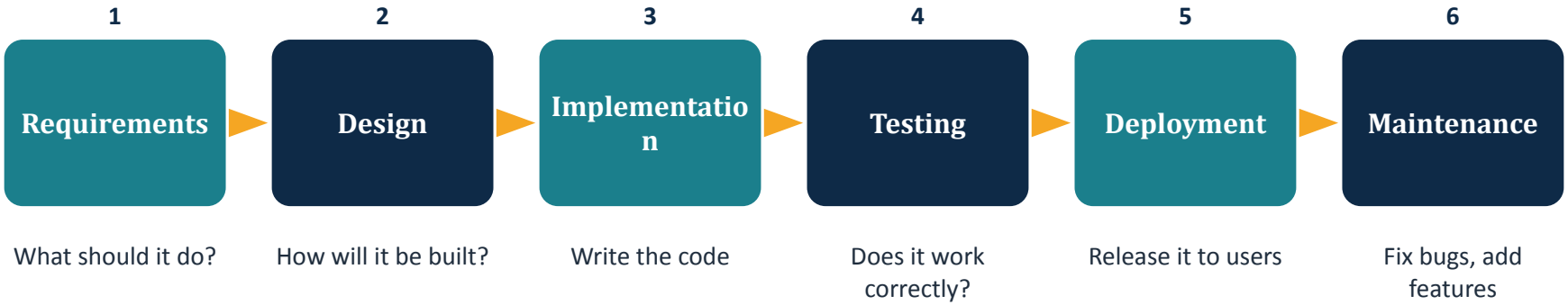
Runs in the background so everything else works: Windows, macOS, Android.



It's all just instructions

Software = a set of instructions (code) that tells a computer exactly what to do, step by step.

What is Software Engineering?



Software Engineering is the discipline of designing, building, testing, and maintaining software — in an organised, reliable way (not just “writing some code”).

Why Software Engineering Matters



Banking apps

A bug could transfer the wrong amount of money to the wrong person.



Flight booking systems

A crash during booking could mean a lost seat — or a lost flight.



Hospital systems

Patient records must be accurate, available, and secure at all times.



Everywhere else

Cars, elevators, traffic lights — all run on software that must not fail.

Quick Check!

Q 1 Name the 6 phases of the software lifecycle.

A: Requirements, Design, Implementation, Testing, Deployment, Maintenance

Q 2 Is Instagram “software”? Why?

A: Yes — it's a program (set of instructions) that runs on your phone

What is an Information System?

Not just software

An Information System (IS) combines Hardware, Software, Data, People, and Process to produce useful information.

Example: a school portal

Hardware (servers) + Software (the website) + Data (grades) + People (students, staff) + Process (how grades get entered) = one Information System.

The 5 Components of an Information System



Hardware

Computers, servers, phones — the physical machines.



Software

The programs that run on the hardware.



Data

The raw facts: names, numbers, dates, transactions.



People

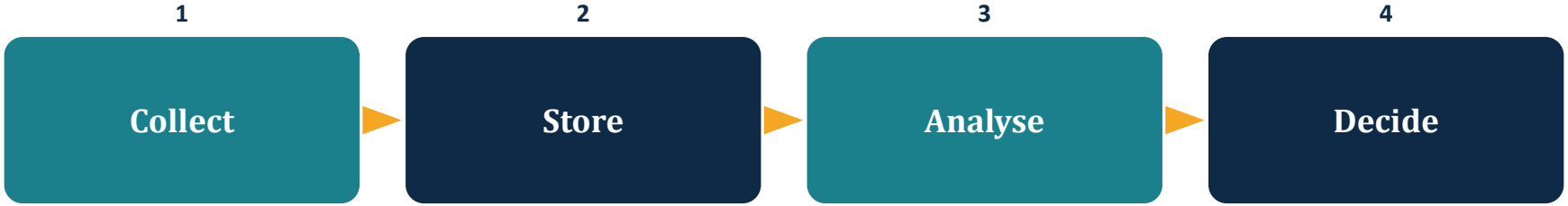
Everyone who uses or maintains the system.



Process

The steps followed to get work done.

Where Does Data Science Fit In?



An Information System gathers data (orders, clicks, sensors...)

Data is saved in files or databases

We use Python to explore and understand it

People make better decisions from the results

This entire course is about step 3: using Python to turn raw data into understanding.



LET'S DISCUSS

Can you name one Information System you used today?

An ATM, a school portal, a food delivery app, Instagram...

Quick Check!

Q 1 Name the 5 components of an Information System.

A: Hardware, Software, Data, People, Process

Q 2 In a food delivery app, what would be an example of “Data”?

A: Your order, your address, the restaurant's menu prices

Why Python?



Reads like English

print("Hello") — you can almost guess what code does.



Huge community

Millions of learners and free help online for almost any problem.



Extremely versatile

Data science, websites, automation, AI, games — one language, many uses.



Beginner-friendly

Designed to be readable — a great first programming language.

Python vs. Other Languages

Java

```
public class Hello {
  public static void main(String[] a)
{
    System.out.println( "Hello!");
  }
}
```

- 4 lines just to print one message

Python

```
print("Hello!")
```

- 1 line, no setup needed
- This is why Python is a favourite first language

Where Is Python Used Today?



Data Science

Analyse data and build charts — this course!



Web Development

Instagram and Spotify use Python on their servers.



AI & Machine Learning

ChatGPT-style tools are largely built and trained with Python.



Automation

Scripts that do boring, repetitive tasks automatically.



Games

Simple games (like the one we'll build today!) and prototypes.

How Does Python Actually Run?



Unlike some languages, Python doesn't need a separate “compile” step — you write and run instantly. That's why notebooks (Jupyter / Colab) feel so interactive.

Variables: Storing Information

```
name = "Amine"
age = 20
price = 19.90
is_student = True

print(name, age, price, is_student)
```

- A variable is a **named box** that holds a value
- No need to declare a type — Python figures it out!
- Use `=` to assign a value
- Names should be clear: `age`, not `a`

The 4 Basic Data Types

Type	Meaning	Example
`int`	Whole number	20, -5, 1000
`float`	Decimal number	19.90, 3.14, -0.5
`str`	Text (string)	"Amine", "Hello!"
`bool`	True or False	True, False

Use the built-in type() function to check any variable's type.

Predict the Output!

```
x = 7
y = "7"
z = 3.5

print(type(x))
print(type(y))
print(type(z))
print(x == int(y))
```

```
<class 'int'>
<class 'str'>
<class 'float'>
True
```

- Try to guess the output **before** we reveal it!
- `7` and `"7"` look similar but are **different types**
- `int(y)` converts the string "7" into the number 7

print(): Showing Output

```
print("Hello, world!")
print("Age:", 20)
print("A", "B", "C", sep=" - ")
print("Line 1", end=" -> ")
print("Line 2")

name = "Lina"
print(f"Welcome, {name}!")
```

- `print()` displays text on the screen
- Separate multiple values with commas
- `sep=` changes the separator, `end=` changes the line ending
- An **f-string** (`f"...{variable}..."`) inserts a variable into text

input(): Getting Information from the User

```
name = input("What is your name? ")
print("Hello, " + name + "!")

age_text = input("How old are you? ")
print(type(age_text))     # <class 'str'>  !

age = int(age_text)
print("Next year you'll be", age + 1)
```

- `input()` pauses the program and waits for the user to type
- **Important:** `input()` always returns a **string**, even for numbers!
- Convert it with `int(...)` or `float(...)` before doing maths

Predict the Output! (the input() trap)

```
a = "5"
b = "3"
print(a + b)

c = int(a) + int(b)
print(c)
```

```
53
8
```

- What does `"5" + "3"` give you — 8, or something else?
- Adding two strings **joins them together** (concatenation)
- Convert first if you want real addition!

Arithmetic Operators

Operator	Meaning	Example → Result
`+ - *`	Add, subtract, multiply	5 + 2 → 7
`/`	Division (always gives a float)	7 / 2 → 3.5
`//`	Integer (floor) division	7 // 2 → 3
`%`	Modulo — the remainder	7 % 2 → 1
`**`	Power (exponent)	2 ** 3 → 8

The % operator is very useful — for example, to check if a number is even: n % 2 == 0

Comparison & Logical Operators

Operator	Meaning	Example → Result
`== !=`	Equal to / not equal to	5 == 5 → True
`> < >= <=`	Greater / less than (or equal)	7 > 10 → False
`and`	Both conditions must be true	(5 > 1) and (2 > 1) → True
`or`	At least one must be true	(5 > 1) or (2 > 100) → True
`not`	Reverses True/False	not (5 > 1) → False

Note: use == to compare values. A single = is only for assigning a variable!

Predict the Output!

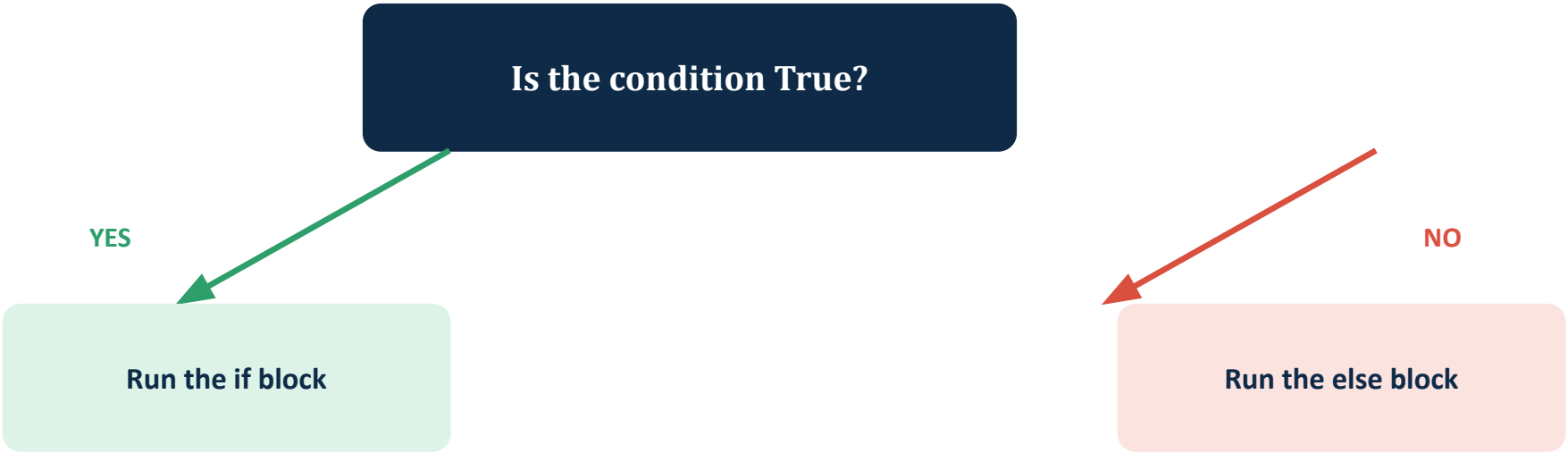
```
age = 17
has_ticket = True

print(age >= 18)
print(age >= 18 or has_ticket)
print(age >= 18 and has_ticket)
```

```
False
True
False
```

- Work through each line **before** checking the output
- Remember: `or` needs only one True; `and` needs both

if / else: Making Decisions




```
age = 17
if age >= 18:
    print("You can vote!")
else:
    print("Not yet — almost there!")
```

if / elif / else: More Than Two Paths

```
temperature = 8

if temperature > 30:
    print("It's hot!")
elif temperature > 15:
    print("Nice weather.")
elif temperature > 0:
    print("A bit cold.")
else:
    print("Freezing!")
```

```
A bit cold.
```

- Python checks each condition **top to bottom**
- The first True condition runs — the rest are skipped
- `else` catches everything else

Quick Check!

Q 1 What prints if score = 55, using if score>=60 ("Pass") elif score>=50 ("Retake") else ("Fail")?

A: "Retake" — the second condition is the first one that's True

Q 2 What symbol compares two values for equality?

A: == (double equals sign)

for Loops: Repeating Actions

```
for i in range(5):
    print(i)

fruits = ["apple", "banana", "cherry"]
for f in fruits:
    print("I like", f)
```

```
0
1
2
3
4
I like apple
I like banana
I like cherry
```

- `range(5)` gives 0, 1, 2, 3, 4 — **5 numbers, starting at 0**
- A `for` loop can also walk through a list directly
- The indented lines run **once per item**

Two Very Useful for-Loop Patterns

Multiplication table

```
n = 7
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

- range(1, 11) → 1 up to 10

Sum of numbers

```
total = 0
for i in range(1, 101):
    total += i     # adds i each time

print(total)
```

- += adds to the running total
- Result: 5050

Predict the Output!

```
total = 0
for i in range(1, 5):
    total = total + i
    print("i =", i, "total =", total)
```

```
i = 1 total = 1
i = 2 total = 3
i = 3 total = 6
i = 4 total = 10
```

- Trace through the loop by hand, one line at a time
- `range(1, 5)` → 1, 2, 3, 4 (stops **before** 5)

Lists: An Ordered Collection

```
fruits = ["apple", "banana", "cherry"]

print(fruits[0])      # apple  (indexing starts at 0!)
print(fruits[-1])     # cherry (last item)
print(len(fruits))    # 3

fruits.append("mango")
print(fruits)
```

- A list holds **many values in one variable**, in order
- Indexing starts at **0**, not 1
- `append()` adds an item to the end
- `len()` gives the number of items

List Cheat-Sheet

Operation	Code	Result
Access by position	`fruits[1]`	"banana"
Add an item	`fruits.append("kiwi")`	adds to the end
Change an item	`fruits[0] = "pear"`	replaces "apple"
Length	`len(fruits)`	number of items
Membership test	`"kiwi" in fruits`	True or False
Loop through it	`for f in fruits:`	visits each item

Tuples: Like Lists, but Locked

List — can change

```
colors = ["red", "green"]
colors[0] = "blue"      # OK!
colors.append("yellow") # OK!
```

- Use when the data may change

Tuple — cannot change

```
point = (3, 7)
# point[0] = 5    # ERROR!

x, y = point    # unpacking
print(x, y)
```

- Use for fixed data: coordinates, RGB colours
- Parentheses () instead of brackets []

Dictionaries: Key → Value Pairs

```
student = {
    "name": "Salma",
    "age": 21,
    "grade": "A",
}

print(student["name"])
student["city"] = "Tunis"       # add a new key

for key, value in student.items():
    print(key, "->", value)
```

- A dictionary stores data as **key: value** pairs
- Access a value with `dict["key"]`, not a number
- `.items()` lets you loop through both keys and values at once

Quick Check!

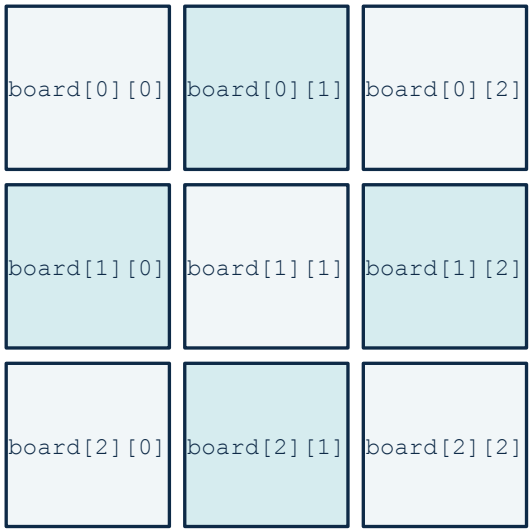
Q 1 fruits = ["a","b","c"] — what is fruits[1]?

A: "b" (indexing starts at 0)

Q 2 Which type would you use for a fixed (x, y) coordinate?

A: A tuple — because it shouldn't change

Lists Inside Lists: a Grid!




```
board = [
    ["_", "_", "_"],
    ["_", "_", "_"],
    ["_", "_", "_"],
]

print(board[1][2])    # row 1, column 2
```

- A list can contain **other lists**
- This creates rows and columns — a **grid**
- `board[row][col]` reaches one cell
- This is exactly how we'll build our Tic-Tac-Toe board!

Reading and Changing a Nested List

```
board = [
    ["_", "_", "_"],
    ["_", "_", "_"],
    ["_", "_", "_"],
]

board[1][1] = "X"        # place X in the centre

for row in board:
    print(row)
```

```
['_', '_', '_']
['_', 'X', '_']
['_', '_', '_']
```

- `board[1]` is the whole **middle row** (a list)
- `board[1][1]` is one **cell** inside that row
- Looping with `for row in board:` visits each row

Quick Check!

Q 1 board = [["_","_"],["_","_"]] — how do you set the top-right cell to "O"?

A: board[0][1] = "O"

Q 2 What does `for row in board:` give you each time?

A: One whole row of the grid (a list)

What Is a Library?

A toolbox of ready-made code

Someone already wrote and tested useful functions — why rewrite them?

Example: random

Generate random numbers, shuffle a list, pick a random item.

Example: math

Square roots, rounding, trigonometry, and more.

The import Statement

```
import random

number = random.randint(1, 100)
print(number)

import math
print(math.sqrt(16))          # 4.0

from math import pi
print(pi)                     # 3.14159...
```

- **`import library_name`** loads the whole library
- Use **`library_name.function()`** to call its tools
- **`from library import thing`** imports just one piece directly

A Few Libraries You'll Meet Soon



random

randint(), choice(), shuffle()



math

sqrt(), floor(), pi



datetime

Work with dates and times



pandas, matplotlib...

Coming in the next chapters!

Quick Check!

Q 1 What line of code loads the random library?

A: import random

Q 2 Why use a library instead of writing everything yourself?

A: It's tested, reliable code that already exists — saves time

Today's Lab: Practice + a Real Game!



Hands-on lab

Lab 0 — Python Basics & Tic-Tac-Toe

Duration: ~1.5 h

You'll practise every tool from today, then combine them all into a real two-player game.

- print() and input()
- Variables and simple maths
- if / else decisions
- for loops
- Lists, tuples, dictionaries
- Importing a library
- CHALLENGE: build Tic-Tac-Toe!

Don't skip ahead — each exercise builds a skill you'll reuse in the challenge.

The Challenge: Tic-Tac-Toe



Two players, X and O, take turns.

Everything is built from what you learned today:

- **A nested list** stores the 3x3 board
- **input()** asks each player for a row and column
- **if / else** checks the cell is empty before playing
- **a for loop** runs the game for up to 9 turns

By the end of the lab, you will have built a real, playable game from scratch!

Key Takeaways

- Software Engineering = building software in an organised, reliable way
- An Information System combines Hardware, Software, Data, People, Process
- Python is readable, versatile, and beginner-friendly
- input() always returns text — convert it before doing maths
- if / elif / else picks one path; for loops repeat actions
- Lists, tuples, and dictionaries store data in different ways
- import lets you reuse code that others already wrote and tested

Final Recap Quiz

Q 1 What does len([1,2,3,4]) return?	**A:** 4
Q 2 Which loop keyword repeats an action a set number of times?	**A:** for
Q 3 What type does input() always return?	**A:** A string (str)
Q 4 Which data type cannot be changed after creation?	**A:** A tuple
Q 5 What key word starts a decision structure?	**A:** if
Q 6 What statement do we use to reuse someone else's code?	**A:** import



Questions?

Let's open the lab and start coding!